

NLP Project (bozza)

Introduction

Exploratory Data Analysis

Performing a Comprehensive Descriptive Analysis

Class Distribution Analysis

Text Feature Analysis

Vocabulary Analysis

Text Preprocessing Methodology

Feature Engineering

Analysis of N-grams (uni-, bi-, and tri-grams)

Extraction of Stylometric Features:

Syntactic Complexity

Sentiment

Emotional Load

Sentence Length

Embedding Techniques

Word2Vec

GloVe

FastText

Classical Classification Models

Support Vector Machine (SVM)

Random Forest

Gradient Boosting

Naive Bayes

Logistic Regression

Advanced Deep Learning Models

BERT

RoBERTa (additional)

In-depth Analysis

Feature Importance

Detailed Confusion Matrix

Error Analysis

Misclassification Cases

Conclusions

Introduction

This thesis explores Natural Language Processing (NLP) techniques for text classification, specifically focusing on the challenging task of distinguishing between conspiracy theories and critical thinking in online discourse.

The core problem addressed in this research is the automatic detection and classification of conspiracy theories (CTs) in digital text. While CTs are traditionally defined as narratives attributing major events to hidden, malevolent actors, the technical challenge lies in developing NLP models that can accurately differentiate between genuine critical thinking and conspiratorial ideation.

Current research in this field has highlighted a significant gap: existing classification systems often struggle to make this crucial distinction between critical and conspiratorial thinking. This limitation poses real-world implications, as misclassifying critical discourse as conspiratorial could potentially drive legitimate skeptics toward conspiracy communities.

The research utilizes a dataset collected from Telegram, focusing on discussions related to the COVID-19 pandemic in both English and Spanish (we will focus on English dataset).

The primary objective of this thesis is to develop and evaluate binary classification models that can effectively distinguish between conspiratorial and critical texts. This work contributes to the broader field of content moderation and digital discourse analysis, addressing a key challenge presented at PAN 2024: the accurate identification and classification of conspiracy narratives while preserving legitimate critical discourse.

Exploratory Data Analysis

The classification task encompasses a binary distinction between two categories of messages in the public health domain:

1. Critical messages: These express reasoned questioning of major public health decisions while maintaining a non-conspiracist perspective.
2. Conspiratorial messages: These interpret pandemic-related events and public health measures as orchestrated by clandestine, influential entities with malevolent intent.

- Dataset Composition: The corpus consists of textual entries categorically labeled as either CONSPIRACY or CRITICAL

The dataset follows a structured format detailed below, which facilitates efficient natural language processing and machine learning applications:

```
{
  "id": "91221",
  "text": "\" Scientistism backed ... not \" \" science - backed \" \" . There is nothing sci
entific about the Covid or childhood vaccine quackery that is slowly but surely killing , maiming and neur
ologically injuring the next generation – just as the Clintons and other Moloch worshippers want . \" \" ,
  "category": "CONSPIRACY",
  "annotations": [
    {
      "span_text": "that is slowly but surely killing , maiming and neurologically injurin
g the next generation",
      "category": "NEGATIVE_EFFECT",
      "start_char": 128,
      "end_char": 219,
    },
    ...
  ]
}
```

The dataset is structured as a JSON file, where each entry contains two key fields: a unique identifier ("id") and a categorical label ("category") denoting whether the text is classified as "CRITICAL" or "CONSPIRACY". This standardized format facilitates efficient data processing and model training.

```
[
  {
    "id": "6742",
    "category": "CRITICAL"
  },
  {
    "id": "10596",
    "category": "CONSPIRACY"
  },
  ...
]
```

Performing a Comprehensive Descriptive Analysis

I developed a comprehensive exploratory data analysis script to examine the provided dataset in detail.

`dataset_analyze.py`

The analysis pipeline generates various visualizations and statistical measures to characterize the dataset's properties:

- **Class Distribution Analysis:**
 - Statistical metrics and visualization of class balance through bar charts, essential for understanding potential dataset biases.
- **Textual Feature Analysis:**
 - Computation of descriptive statistics for text length and token count distributions.
 - Distribution visualization through boxplots to identify potential outliers and patterns.
- **Vocabulary Analysis:**
 - Quantitative assessment of total and unique token frequencies per class, providing insights into lexical diversity.

Class Distribution Analysis

Purpose: Visualizes class frequencies in the dataset using bars, helping identify potential dataset imbalances that could affect model training.

Class Distribution in Train Set:

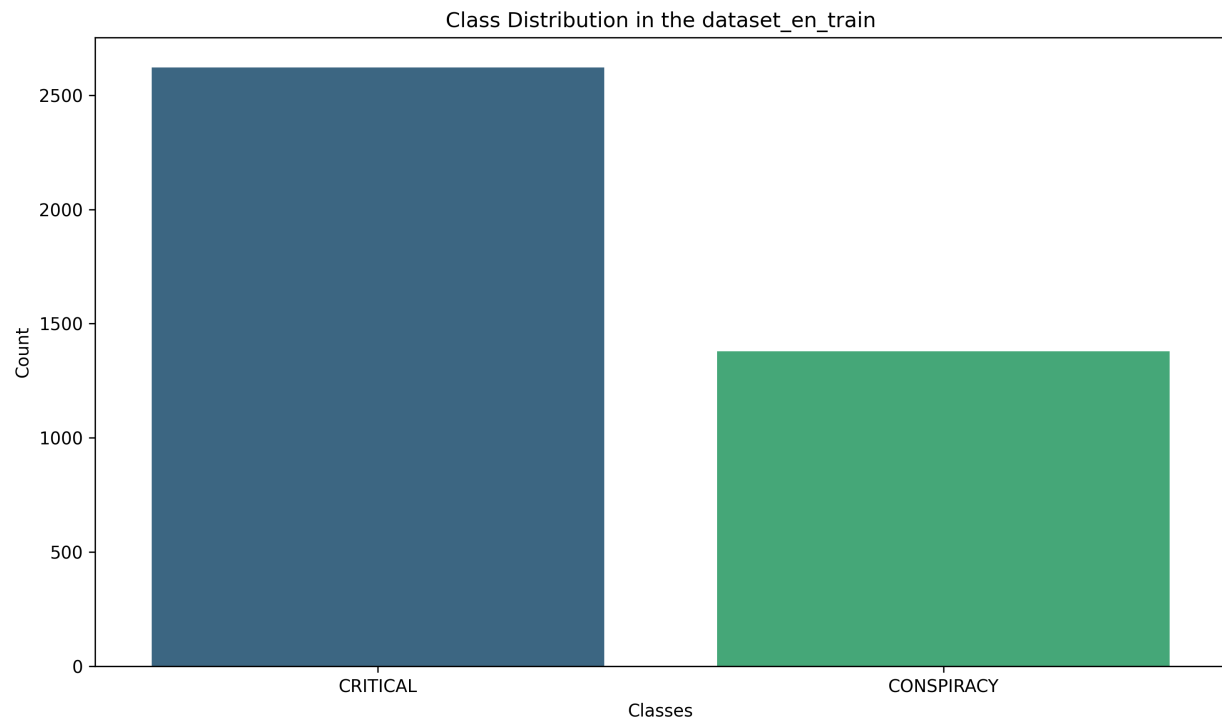
Dataset Size: 4000

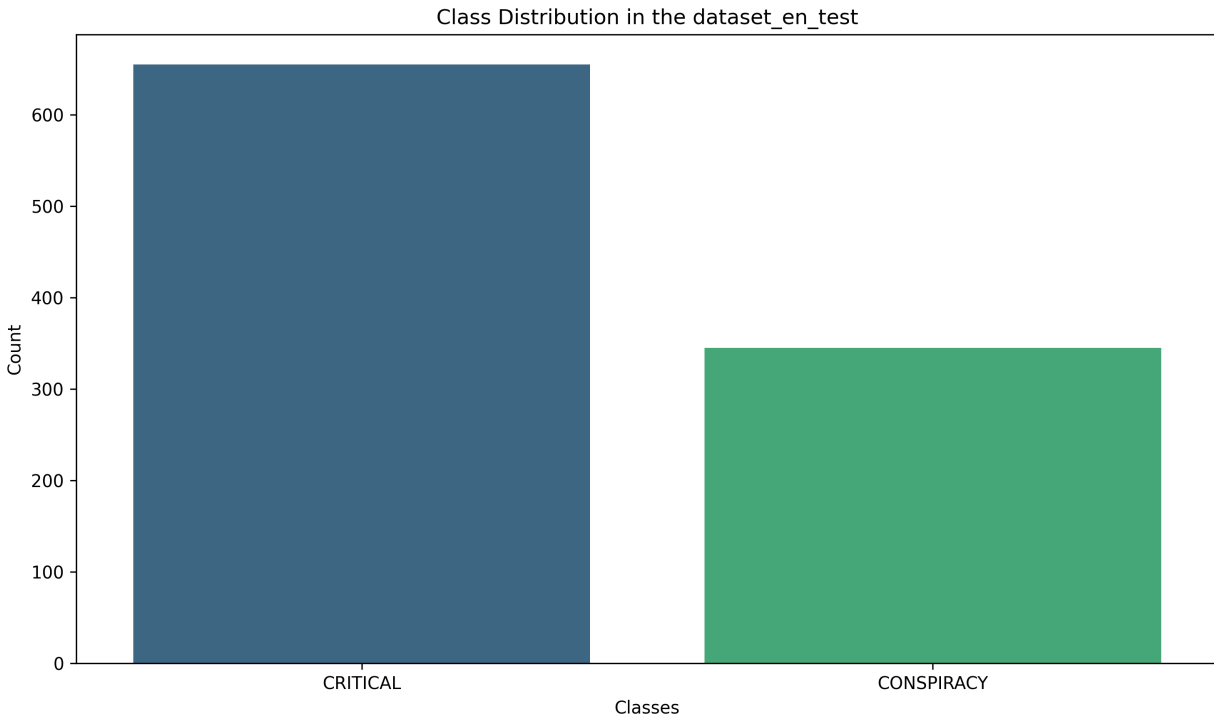
Class	Count	Percentage
CRITICAL	2621	65.525%
CONSPIRACY	1379	34.475%

Class Distribution in Test Set:

Test set size: 1000

Class	Count	Percentage
CRITICAL	655	65.5%
CONSPIRACY	345	34.5%





Text Feature Analysis

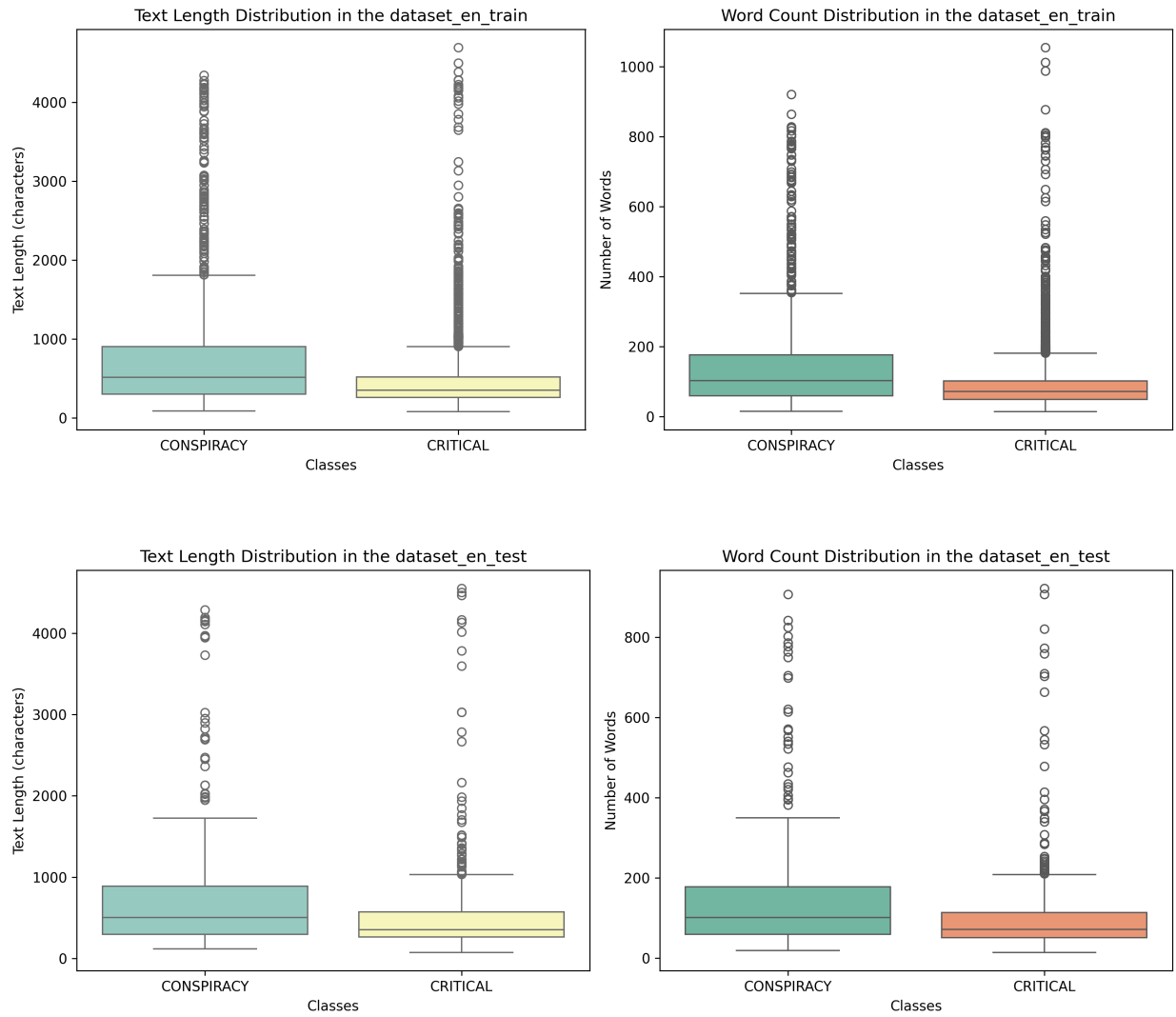
Purpose: Shows text length and word count distributions per class using boxplots, highlighting median, quartiles, and outliers to identify text characteristic patterns.

- CONSPIRACY texts have higher median length than CRITICAL texts
- CONSPIRACY texts show greater length variability
- Both categories have outliers, with CRITICAL showing more extreme ones
- Both distributions are positively skewed, especially CRITICAL

Vocabulary Analysis

Purpose: Presents unique word counts per class in a table format, indicating linguistic variety and potential preprocessing needs.

- Similar pattern to text length distribution
- CONSPIRACY texts tend to use more words
- Presence of outliers in both categories, with some texts containing a significantly higher number of words



Text Preprocessing Methodology

Before conducting N-gram analysis and implementing various classification models, we employ a comprehensive text preprocessing pipeline to standardize and clean the raw textual data.

The script performs a detailed preprocessing of the text data by carrying out the following steps:

- 1. Unicode to ASCII Conversion:** It normalizes text by converting any non-ASCII characters to their closest ASCII equivalents. This ensures uniformity and helps in handling text from diverse character encodings.

2. **Lowercasing:** The text is transformed to lowercase to maintain consistency and avoid treating the same word in different cases (e.g., "Apple" vs "apple") as distinct entities.
3. **Stopword Removal:** Common but non-informative words such as "the," "and," and "is" are removed from the text, as they don't contribute to the meaning and can introduce noise in further analysis.
4. **Punctuation Removal:** All punctuation marks (e.g., periods, commas, exclamation points) are stripped from the text, as they typically don't add significant value for tasks like sentiment analysis or classification.
5. **Removal of Single-Letter Words:** Words consisting of only a single letter (e.g., "a," "I") are discarded, as they rarely carry meaningful information in most text mining applications.
6. **Porter Stemming:** The script applies the Porter Stemming algorithm, which reduces words to their root forms (e.g., "running" to "run") in order to standardize variations of words and minimize the impact of inflections on analysis.

```
preprocess.py
```

The preprocessing pipeline ingests JSON-formatted data from trainset (`dataset_en_train.json`) and testset(`dataset_en_test.json`), applies the aforementioned cleaning procedures, and outputs a standardized dataset optimized for subsequent natural language processing analysis and machine learning model development.

Sample Input Data:

THIS IS MASSIVE Australian Senator Malcolm Roberts exposes nanotech found in the Covid ' vaccines ' and says they are genocide . He is the first politician to expose this ! SHARE [https :// t . me / LauraAbolichannel](https://t.me/LauraAbolichannel) "

Sample Output data:

*massiv australien senat malcolm robert expos nanotech found
covid vaccin say genocid politician expos share
lauraabolichannel*

Feature Engineering

Analysis of N-grams (uni-, bi-, and tri-grams)

For the n-grams analyze i created a python script

[n_grams.py](#)

Both "CONSPIRACY" and "CRITICAL" categories show a strong concentration on key themes:

- "Vaccin" (vaccine) is the most frequent term overall
- "Covid" and "Covid 19" are consistently present
- References to people, health, and global dimensions

1. Structural Differences:

- CONSPIRACY Category:
 - More emphasis on concepts like "deep state", "world order"
 - Presence of terms such as "new world order", "big pharma"
 - More complex and elaborate theoretical language
- CRITICAL Category:
 - More technical and medical focus
 - Greater references to data ("death", "health", "pfizer")

- More mentions of scientific sources (NCBI, NLM, NIH)
1. Significant N-grams:
CONSPIRACY:
 - Bigrams like "deep state", "world order"
 - Trigrams such as "new world order", "biolog chemic weapon"CRITICAL:
 - More technical bigrams like "covid vaccin", "vaccin passport"
 - Data-oriented trigrams like "total death injuri"
 2. Data Distribution:
The data comes from two sets (train and validation), with consistent numbers between them, suggesting stability in the analysis.
 3. Interesting Observations:
 - Both categories heavily discuss vaccines and COVID-19
 - "AgentsOfTruth" appears in both datasets, potentially indicating a specific information source or platform
 - The critical category seems more grounded in scientific and medical terminology

Extraction of Stylometric Features:

Syntactic Complexity

Sentiment

Emotional Load

Sentence Length

Embedding Techniques

Word2Vec

GloVe

FastText

Classical Classification Models

Support Vector Machine (SVM)

Random Forest

Gradient Boosting

Naive Bayes

Logistic Regression

Compare performance using MCC (Matthews Correlation Coefficient) and F1-score.

Advanced Deep Learning Models

BERT

RoBERTa (additional)

In-depth Analysis

Feature Importance

Detailed Confusion Matrix

Error Analysis

Misclassification Cases

Conclusions